1. Object-Oriented Programming (OOP) Pillars

Don't just define them; explain how they create better systems.

A. Encapsulation

- **Concept:** Wrapping data (variables) and code (methods) together as a single unit and restricting direct access to inner workings.
- **The "Why":** It protects the internal state of an object. You can change the implementation logic without breaking the code that *uses* the object.
- **Real-World Example:** A BankAccount class.
    - **Bad:** account.balance = -500; (Direct access allows invalid st ate).
    - **Good:** private double balance; with public void withdraw(double amount). The method includes validation logic (e.g., if (amount > balance) throw error).
- **Interview Question:** *How do you achieve encapsulation in Java?*
    - **Answer:** By declaring class variables as private and providing public getter and setter methods to access/modify them.

B. Abstraction

- **Concept:** Hiding complex implementation details and showing only the essential features of the object.
- **The "Why":** It reduces complexity. The user cares *what* the object does, not *how* it does it.
- **Real-World Example:** A DatabaseConnection interface.
    - You call connection.connect(). You don't care if the underlying code is talking to MySQL, PostgreSQL, or Oracle. The "connect" concept is abstract.
- **Code:** Abstract classes (abstract class) and Interfaces (interface).

C. Inheritance

- **Concept:** One class acquires the properties (fields) and behaviors (methods) of another class.
- **The "Why":** Code Reusability.
- **Key Rule:** Java supports **Single Inheritance** only (a class can extend only one parent class) to avoid the "Diamond Problem" (ambiguity if two parents have the same method).

- **Interview Question:** *Why doesn't Java support multiple inheritance?*
    - **Answer:** To prevent ambiguity. However, a class can implement *multiple interfaces*.

D. Polymorphism

- **Concept:** "Many forms." The ability of an object to take on many forms.
- **Types:**
    1. **Compile-time (Static):** Method Overloading (Same method name, different parameters).
        - *Example:* add(int a, int b) vs add(double a, double b).
    2. **Runtime (Dynamic):** Method Overriding (Same method signature in Parent and Child class).
        - *Example:* Animal generic = new Dog(); generic.sound(); (Calls Dog's sound).

---

2. Abstract Class vs. Interface (The Classic Question)

Feature	Abstract Class	Interface
Methods	Can have both abstract (no body) and concrete (with body) methods.	Until Java 8, only abstract. Java 8+ allows default and static methods.
Variables	Can have private, protected, etc. (Stateful).	Variables are always public static final (Constants).
Inheritance	A class can extend only **ONE** abstract class.	A class can implement **MULTIPLE** interfaces.
Constructor	Can have constructors.	Cannot have constructors.
Usage	Use when related classes share code (e.g., Employee -> Manager, Developer).	Use to define a contract/capability across unrelated classes (e.g., Serializable, Runnable).

---

3. The "Object" Class Methods

Every class in Java inherits from java.lang.Object. You must know these methods.

1. **equals() vs ==**:
    - ==: Compares **memory references** (address). Are they pointing to the exact same object?
    - equals(): Compares **content** (if overridden).
    - *Note:* If you don't override equals(), it defaults to ==.
2. **hashCode()**:
    - Returns an integer representation of the object's memory address.
    - **Contract:** If obj1.equals(obj2) is true, their hashCode() **MUST** be the same.
3. **toString()**: Returns string representation. Always override this for logging/debugging.

---

4. String Concepts (Very Important)

- **Immutability:** Once a String object is created, it cannot be changed.
    - *Why?* Security (used for DB connections, file paths), Caching (String Pool), and Thread Safety.
- **String Constant Pool:** A special memory area in the Heap.

Java

String s1 = "Hello"; // Goes to Pool

String s2 = "Hello"; // Points to same object in Pool (Memory efficient)

String s3 = new String("Hello"); // Forces a NEW object in Heap (bypasses Pool check)

- **StringBuilder vs. StringBuffer:**
    - String: Immutable (Slow for modifications).
    - StringBuffer: Mutable + **Synchronized** (Thread-safe, but slower).
    - StringBuilder: Mutable + **Not Synchronized** (Fastest, standard for non-threaded string manipulation).

---

5. Java Collections Framework

A. List vs. Set

- **List (ArrayList, LinkedList):** Ordered collection, allows duplicates.
- **Set (HashSet, TreeSet):** Unordered, **unique** elements only (no duplicates).

B. ArrayList vs. LinkedList

- **ArrayList:** Backed by a dynamic **Array**.
    - *Fast:* Random access (get(5) is O(1)).
    - *Slow:* Insert/Delete in middle (needs shifting elements).
- **LinkedList:** Backed by a **Doubly Linked List**.
    - *Fast:* Insert/Delete (just change pointers).
    - *Slow:* Random access (must traverse from head/tail).

C. HashMap Internals (The #1 Interview Topic)

- **Structure:** Array of "Buckets" (Nodes).
- **How put(K, V) works:**
    1. Calculates hash(K).
    2. Finds index: index = hash & (n-1).
    3. If bucket is empty, insert Node.
    4. If bucket has a Node (Collision), check equals(K). If true, update Value. If false, append to Linked List.
- **Java 8 Improvement:** If the Linked List grows too long (Threshold > 8), it converts to a **Balanced Tree (Red-Black Tree)** to improve performance from O(n) to O(log n).

---

6. Exception Handling

- **Hierarchy:** Throwable -> Error (e.g., OutOfMemory - fatal) & Exception.
- **Checked vs. Unchecked:**
    - **Checked:** Inherits from Exception. Compiler forces you to handle it (try-catch or throws). Example: IOException, SQLException. Used for recoverable errors.
    - **Unchecked:** Inherits from RuntimeException. Compiler ignores it. Example: NullPointerException, ArrayIndexOutOfBounds. Usually programming errors.

- **finally block:** Always executes, regardless of exception. Used for closing resources (DB connections, Streams).
- **try-with-resources (Java 7+):** Automatically closes resources implementing AutoCloseable.

Java

```
try (BufferedReader br = new BufferedReader(new FileReader(path))) {
   return br.readLine();
} // br is automatically closed here
```

---

7. Java 8 Features (Modern Java)

- **Lambda Expressions:** Anonymous functions. Reduces boilerplate code.
    - Example: (x, y) -> x + y
- **Functional Interface:** Interface with exactly **one abstract method**. Examples: Runnable, Callable, Comparator.
    - Annotation: @FunctionalInterface.
- **Streams API:** For processing collections of objects (Functional style).
    - *Operations:* filter, map, reduce, collect.
    - *Example:*

Java

```
List<String> names = list.stream()
   .filter(s -> s.startsWith("A"))
   .map(String::toUpperCase)
   .collect(Collectors.toList());
```

- **Optional Class:** A container object which may or may not contain a non-null value. Eliminates NullPointerException.
    - Optional.ofNullable(user).ifPresent(u -> System.out.println(u.getName()));

---

8. Memory Management Basics

- **Stack Memory:**
    - Stores **method execution frames** and **local variables** (primitives and references).
    - Thread-safe (each thread has its own stack).
    - Error: StackOverflowError (e.g., infinite recursion).
- **Heap Memory:**
    - Stores **Objects** (new MyClass()) and Instance Variables.
    - Shared by all threads.
    - Managed by **Garbage Collector (GC)**.
    - Error: OutOfMemoryError (Heap full).
- **Garbage Collection:** Automatic process of reclaiming memory by destroying unreachable objects.
    - *System.gc()* suggests execution, doesn't guarantee it.

---

9. SOLID Principles (OO Design)

Briefly memorize these.

1. **S - Single Responsibility:** A class should have one, and only one, reason to change.
2. **O - Open/Closed:** Open for extension, closed for modification.
3. **L - Liskov Substitution:** Subtypes must be substitutable for their base types.
4. **I - Interface Segregation:** Many specific interfaces are better than one general-purpose interface.
5. **D - Dependency Inversion:** Depend on abstractions (Interfaces), not concretions.

1. JVM Architecture & Memory Model (The Internals)

Interviewers ask this to verify you understand where your code actually runs.

A. The Memory Structure (HotSpot JVM)

- **Heap Memory (The Playground):** Where all **Objects** live. It is divided into generations:
    - **Young Generation (Eden + Survivor S0/S1):** Where new objects are born. Most objects die here quickly (e.g., temporary variables in a loop).
    - **Old Generation (Tenured):** Where long-lived objects go (e.g., Cache, Connection Pools) after surviving multiple GC cycles in the Young Gen.
- **Metaspace (Java 8+):** Replaced "PermGen". Stores **Class Metadata** (static variables, method definitions). It uses native OS memory, not Heap.
- **Stack Memory:** Thread-specific. Stores local primitives and method call frames.

B. The ClassLoader Subsystem

- **Bootstrap ClassLoader:** Loads core Java classes (rt.jar -> java.lang.*). Written in C/C++.
- **Extension ClassLoader:** Loads from lib/ext.
- **Application ClassLoader:** Loads **your** code (classpath).
- **Delegation Principle:** A request to load a class is always sent to the parent first. If the parent can't find it, the child tries.
    - *Security:* You cannot override java.lang.String by creating your own class with that name, because Bootstrap loads the real one first.

---

2. Garbage Collection (GC) Algorithms

You don't need to write a GC, but you must know how to tune it.

- **The "Stop-the-World" Event:** All application threads **freeze** while GC runs. Your goal is to minimize this pause time.
- **Minor GC:** Cleans the **Young Generation**. Very fast.
- **Major GC (Full GC):** Cleans the **Old Generation**. Slower. If this happens too often, your app latency spikes.
- **G1GC (Garbage First - Default in Java 9+):**
    - Divides the heap into small regions (like a chessboard).

- It cleans the regions with the *most* garbage first (hence "Garbage First").
- **Advantage:** Predictable pause times (you can set a target, e.g., "Max 200ms pause").

---

3. Advanced Concurrency (Multi-threading)

The hardest topic in Java.

A. volatile Keyword

- **Problem:** CPU Caching.
  - Core 1 runs Thread A. Core 2 runs Thread B.
  - Thread A updates a variable flag = true. It writes it to **Core 1's Cache**, not main RAM immediately.
  - Thread B checks flag. It reads from **Core 2's Cache**, which still says false.
- **Solution:** volatile boolean flag = true;
  - Forces the write **directly to Main Memory (RAM)**.
  - Guarantees **Visibility** (all threads see the latest value).
  - *Note:* It does NOT guarantee **Atomicity** (e.g., count++ is not safe with just volatile).

B. synchronized vs. ReentrantLock

- **synchronized:** Implicit lock. Easy to use. If a thread gets stuck waiting, it waits forever (potential deadlock).
- **ReentrantLock:** Explicit lock.
  - **Feature:** tryLock(timeout) allows you to say "Wait for 1 second, if I can't get the lock, give up." This prevents deadlocks.
  - **Feature:** Fairness policy (give lock to the longest-waiting thread).

C. CompletableFuture (Async Programming)

- Java 8's answer to JavaScript Promises.
- *Scenario:* Fetch User (2s) AND Fetch Orders (3s), then combine.

Java

CompletableFuture<User> userFuture = CompletableFuture.supplyAsync(() -> getUser());

CompletableFuture<Order> orderFuture = CompletableFuture.supplyAsync(() -> getOrder());

// Runs in parallel. Total time = Max(2, 3) = 3s. Not 2+3=5s.

CompletableFuture.allOf(userFuture, orderFuture).join();

---

4. Java 8 Streams Under the Hood

Why are Streams faster/better?

- **Lazy Evaluation:** Intermediate operations (filter, map) do **nothing** until a terminal operation (collect, forEach) is called.
    - *Example:* list.stream().filter(x -> x > 10).map(x -> x * 2).findFirst();
    - If the first element matches, the stream **stops**. It doesn't process the whole list. This is called **Short-Circuiting**.
- **Parallel Streams:** list.parallelStream() automatically splits the collection into chunks and processes them on multiple cores using the **ForkJoinPool**.

---

5. Spring Boot Internals (Brief)

Since you have 3.5 years exp, they assume you know Spring.

- **Dependency Injection (IoC):** The container creates objects (Beans) and injects them where needed. You don't use new Service().
- **Bean Scopes:**
    - **Singleton (Default):** One instance per application. (Stateless).
    - **Prototype:** New instance every time it is requested. (Stateful).
    - **Request:** One per HTTP request.
- **AOP (Aspect Oriented Programming):** How @Transactional works.
    - Spring creates a **Proxy** (a wrapper) around your class.
    - When you call save(), the Proxy starts a transaction -> calls your save() -> commits the transaction.

---

6. Common Design Patterns in Java

A. Singleton (The "One Object" Pattern)

- *Use Case:* Database Connection, Logger, Configuration.
- *Code:* See the "Double-Checked Locking" example in previous notes.

B. Factory Pattern (The "Creator" Pattern)

- *Use Case:* When logic determines which class to instantiate.
- *Example:* NotificationFactory.create("SMS") returns SMSNotification class.

Java

```
public static Notification create(String type) {
   if (type.equals("SMS")) return new SMSNotification();
   if (type.equals("EMAIL")) return new EmailNotification();
   throw new IllegalArgumentException();
}
```

C. Builder Pattern (The "Constructor" Fix)

- *Use Case:* When an object has 10+ parameters (some optional).
- *Example:* Lombok uses this.

Java

```
User u = new User.Builder()
   .setName("Rohit")
   .setAge(25)
   .build(); // Clean and readable
```

7. Java 17 Specifics (Since you list it on your resume)

- **Records:** Concise way to create immutable data carriers.

Java

```
// Instead of class with getters/toString/hashCode/equals
public record User(String name, int id) {}
```

- **Switch Expressions:**

Java

```
String result = switch(day) {
```

```
    case MONDAY, FRIDAY -> "Work";
    case SUNDAY -> "Rest";
    default -> "Unknown";
};
```

- **Sealed Classes:** Restrict which classes can extend your class.

Java

```
public sealed class Payment permits CreditCard, UPI {}
// Only CreditCard and UPI can extend Payment. No other class.
```